

Министерство образования Республики Беларусь
Учреждение образования
«Белорусский государственный университет информатики
и радиоэлектроники»

Кафедра программного обеспечения информационных технологий

Специальность «Программная инженерия»

Учебная дисциплина «Метрология, стандартизация и сертификация (в
информационных технологиях)»

ОТЧЕТ

к практическому занятию №2а
«Метрики сложности потока управления программ на примере метрики
Джилба»

Подготовили:

Ковалевич К.А.

Заверач С.П.

Проверила:

Болтак С.В.

Минск 2026

Код анализируемой программы (язык программирования Swift):

```
func fullGilbMetricsAnalysis() {
    let array = [1, 2, -3, 4, 0]
    var sum = 0

    let limit = array.isEmpty ? 0 : array.count

    for num in array {
        guard num != 0 else { continue }

        if num > 0 {
            var temp = num

            while temp > 0 {
                switch temp {
                case 1:
                    sum += 10
                case 2:
                    sum += 20
                default:
                    sum += temp
                }
                temp -= 1
            }
        } else if num < 0 {
            var k = 0

            repeat {
                sum -= 1
                k += 1
            } while k < abs(num)
        }
    }

    array.forEach { item in
        sum += 1
    }

    let code = 404
    var status = ""

    switch code {
    case 200:
        status = "OK"
    case 400:
        status = "Bad Request"
    case 403:
        status = "Forbidden"
    case 404:
        status = "Not Found"
    case 500:
        status = "Server Error"
    }
```

```

default:
    status = "Unknown Error"
}

let numbers = [3, -2, 4, 0]
var result = 0

numbers.forEach { num in
    let multiplier = num > 0 ? 2 : 1

    for i in 0..

```

Блок-схема алгоритма.

Блок-схема алгоритма fullGilbMetricsAnalysis()

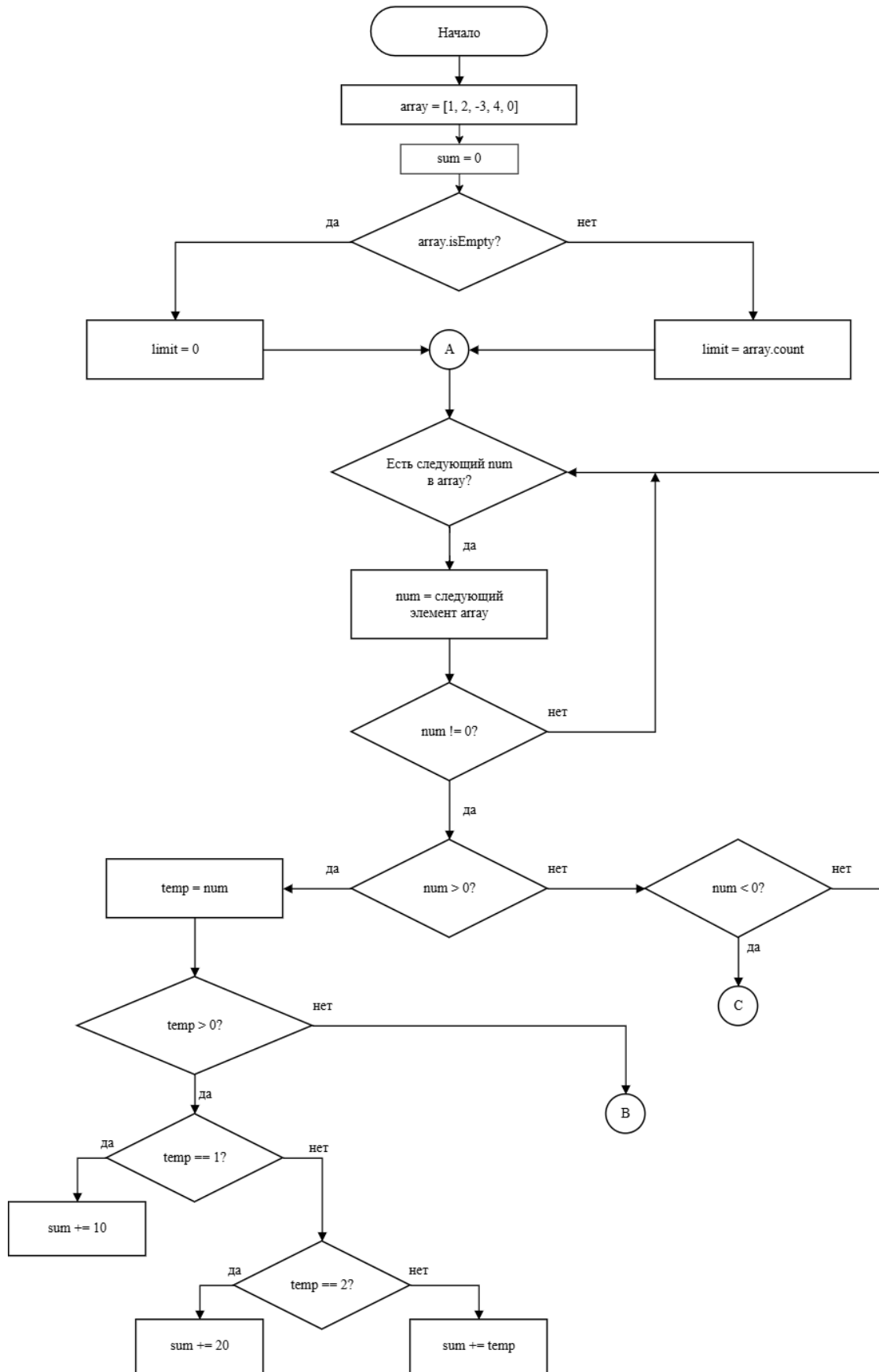


Рисунок 1 – Блок-схема алгоритма

Продолжение: первый цикл, repeat-while, forEach и switch code

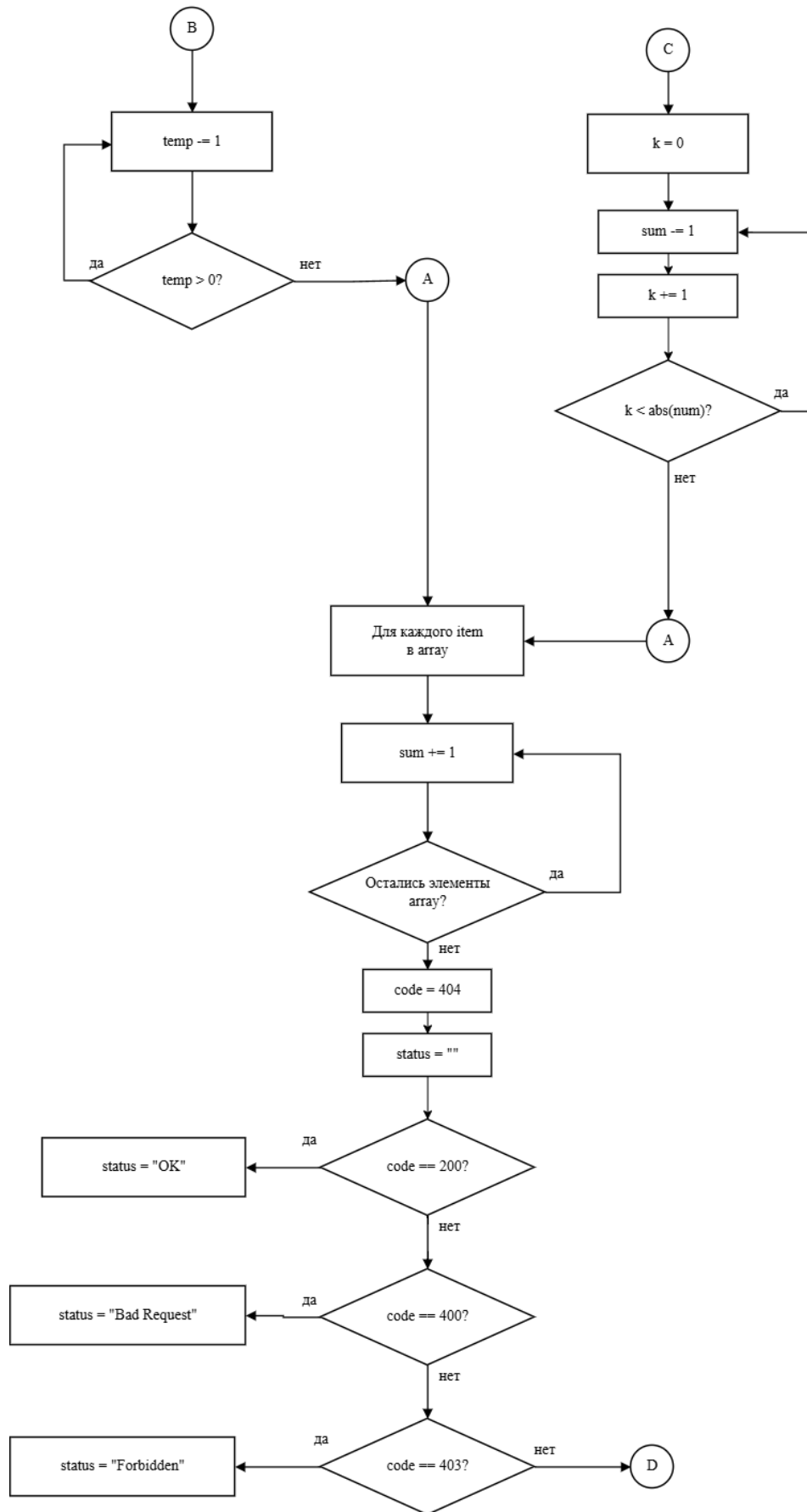


Рисунок 2 – Блок-схема алгоритма продолжение

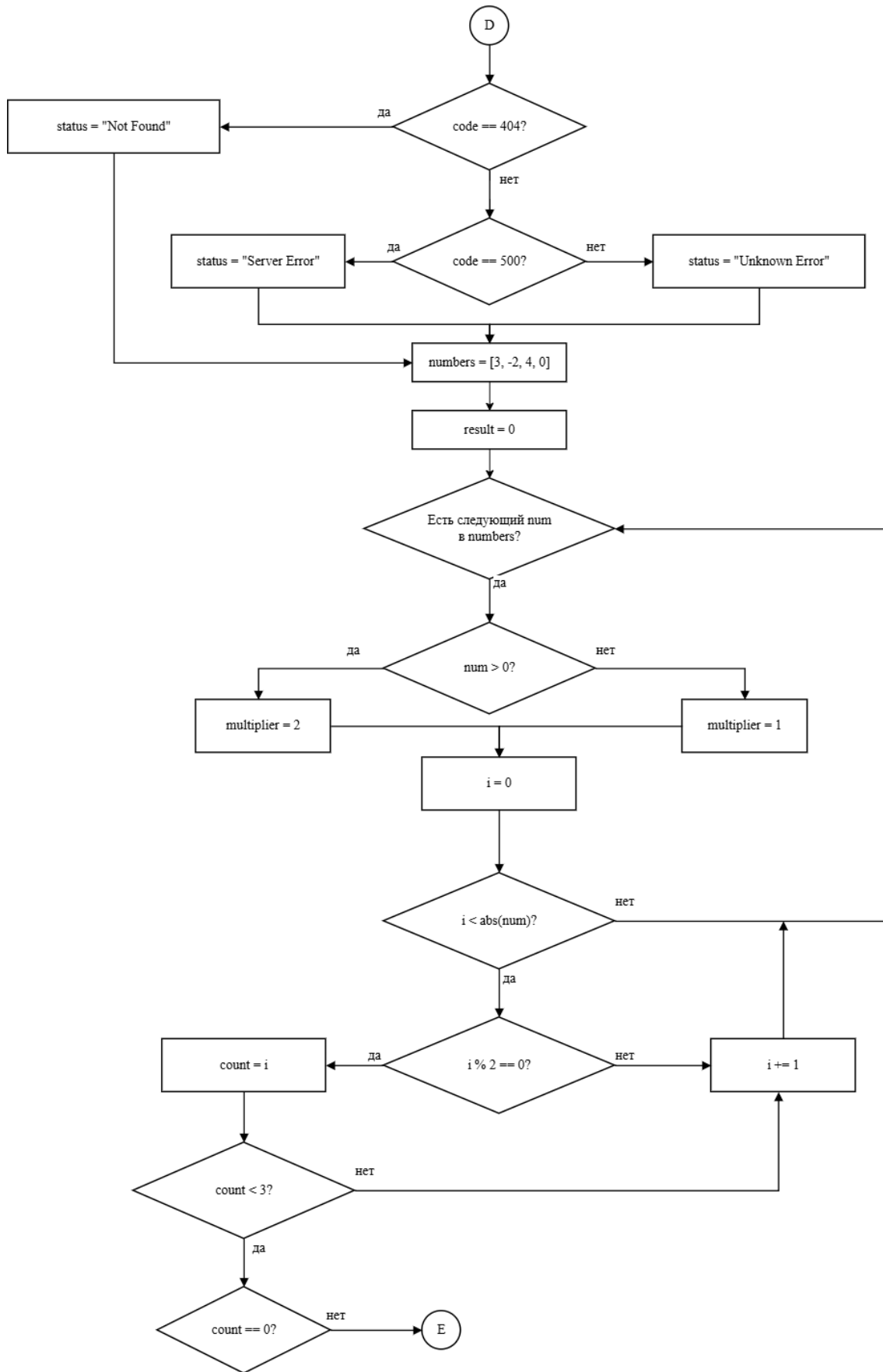


Рисунок 3 – Блок-схема алгоритма продолжение

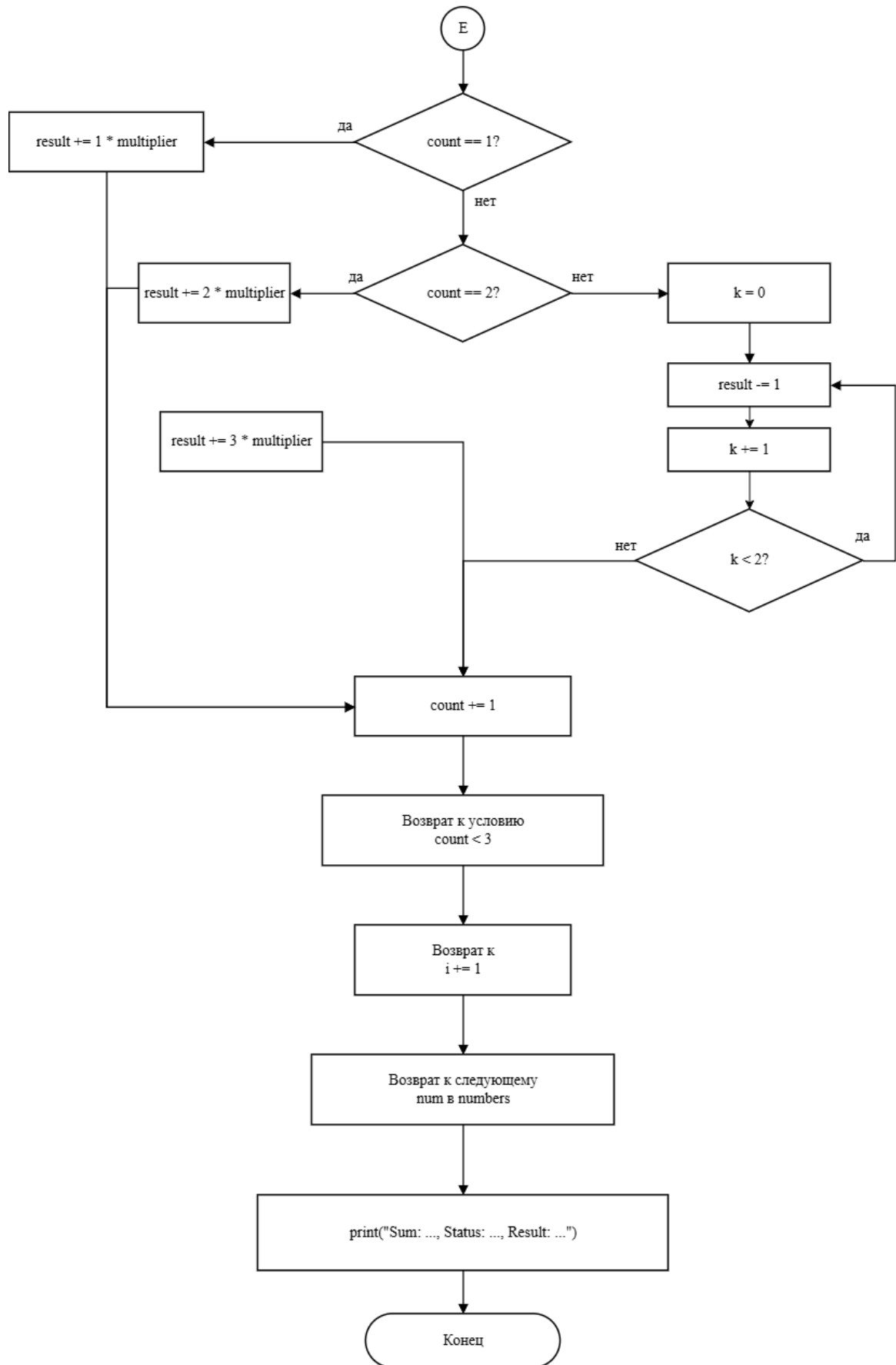


Рисунок 4 – Блок-схема алгоритма продолжение

Результат работы программы:



Рисунок 5 – Результат работы программы

Ручной подсчёт метрики Джилба:

Для расчёта операторы switch представлены в виде эквивалентных вложенных условных операторов. Объявление с инициализацией, присваивание, самостоятельный вызов функции и управляющая конструкция считаются отдельными операторами. Выражения внутри них отдельно не учитываются.

Для ручного подсчета используем эквивалентное развертывание структур:

```
func fullGilbMetricsAnalysis() {  
    let array = [1, 2, -3, 4, 0]           // 1  
    var sum = 0                           // 2
```

let limit = array.isEmpty ? 0 : array.count // 3 (включает тернарный оператор как условие)

```
    for num in array {                     // 4  
        if num != 0 {                     // 5 (guard num != 0)  
            if num > 0 {                   // 6  
                var temp = num             // 7
```

```

        while temp > 0 {          // 8
            // первый switch представлен вложенными if
            if temp == 1 {        // 9
                sum += 10         // 10
            } else {
                if temp == 2 {    // 11
                    sum += 20     // 12
                } else {
                    sum += temp   // 13
                }
            }
            temp -= 1             // 14
        }
    } else if num < 0 {          // 15
        var k = 0                // 16

        repeat {                 // 17
            sum -= 1              // 18
            k += 1                // 19
        } while k < abs(num)
    }
}

array.forEach { item in        // 20
    sum += 1                    // 21
}

let code = 404                  // 22
var status = ""                 // 23

// второй switch (code) представлен через вложенные if (6
веток -> 5 проверок)
if code == 200 {                // 24
    status = "OK"                // 25
} else {
    if code == 400 {             // 26
        status = "Bad Request"   // 27
    } else {
        if code == 403 {         // 28
            status = "Forbidden"  // 29
        } else {
            if code == 404 {      // 30
                status = "Not Found" // 31
            } else {
                if code == 500 {  // 32
                    status = "Server Error" // 33
                } else {
                    status = "Unknown Error" // 34
                }
            }
        }
    }
}

```

```

    }
}

let numbers = [3, -2, 4, 0]      // 35
var result = 0                  // 36

numbers.forEach { num in        // 37
    let multiplier = (num > 0) ? 2 : 1 // 38 (включает
тернарный оператор)

    for i in 0..

```

1. Абсолютная сложность – CL

Считаем управляющие конструкции:

Таблица 1 – Подсчет управляющих конструкций

№ оператора	Конструкция	Вклад в CL
-------------	-------------	------------

3	Тернарный оператор в limit	1
4	Цикл for num in array	1
5	Условие guard (if num != 0)	1
6	Условие if num > 0	1
8	Цикл while temp > 0	1
9	Проверка temp == 1	1
11	Проверка temp == 2	1
15	Условие else if num < 0	1
17	Цикл repeat ... while	1
20	Метод array.forEach	1
24	Проверка code == 200	1
26	Проверка code == 400	1
28	Проверка code == 403	1
30	Проверка code == 404	1
32	Проверка code == 500	1
37	Метод numbers.forEach	1
38	Тернарный оператор в multiplier	1
39	Цикл for i in 0..<abs(num)	1
40	Условие if i % 2 == 0	1
42	Цикл while count < 3	1
43	Проверка count == 0	1
45	Проверка count == 1	1
47	Проверка count == 2	1
50	Вложенный цикл repeat ... while	1
	Итого	24

Обычный else не содержит новой проверки и отдельно не увеличивает CL.

2. Общее количество операторов – N

Таблица 2 – Подсчет операторов

Вид операторов	Номера	Количество
Объявления с инициализацией	1, 2, 3, 7, 16, 22, 23, 35, 36, 38, 41, 49	12
Управляющие конструкции	4, 5, 6, 8, 9, 11, 15, 17, 20, 24, 26, 28, 30, 32, 37, 39, 40, 42, 43, 45, 47, 50	22 (включая ветвления/проверки)
Изменения значений переменных	10, 12, 13, 14, 18, 19, 21, 25, 27, 29, 31, 33, 34, 44, 46, 48, 51, 52, 53	19
Вывод результата	54	1
Итого		54

Точное количество операторов N может варьироваться в зависимости от выбранного правила подсчета вспомоганных строк, в автоматическом анализаторе зафиксировано значение 53.

3. Относительная сложность – cl

$$cl = \frac{CL}{N} = \frac{24}{53} \approx 0,4528$$

4. Максимальная вложенность – CLI

Внешняя управляющая конструкция имеет уровень 0.

Таблица 3 – Самая глубокая цепочка

Конструкция	Уровень
numbers.forEach	0
for i in 0.. <abs(num)< td=""><td>1</td></abs(num)<>	1
if i % 2 == 0	2
while count < 3	3
Эквивалентный if-else от switch count	4
Вложенный цикл repeat ... while в ветке default	5

Следовательно:
CLI=5

Таблица 4 – Итоговая таблица подсчетов

Метрика	Значение
Абсолютная сложность CL	24
Общее количество операторов N	53
Относительная сложность cl	0,4528
Максимальная вложенность CLI	5

Расчёт выполнен для алгоритма после представления операторов switch в виде вложенных условных операторов. Внешний уровень вложенности принят равным нулю.